

camera.py — moduł obsługi kamery

1. Cel modułu

Moduł camera.py odpowiada za przechwytywanie klatek obrazu z kamery i dostarczanie ich pozostałym modułom systemu Stop & Go. Jest pierwszym ogniwem całego łańcucha przetwarzania: kamera → detekcja → odległość → sterowanie.

Moduł rozwiązuje trzy kluczowe problemy inżynierskie:

1. Opóźnienie buforowania — domyślnie OpenCV kolejkuje klatki w buforze; przy wolnym przetwarzaniu system widzi przeszłość, nie teraźniejszość.
2. Blokowanie pętli głównej — bez osobnego wątku każde wywołanie `cap.read()` blokuje program na ~33 ms czekając na nową klatkę.
3. Redundancja danych — obraz BGR (3 kanały) jest trzykrotnie cięższy niż skala szarości; konwersja tutaj odciąża wszystkie kolejne moduły.

2. Importy i ich uzasadnienie

```
import cv2
import threading
```

`cv2` — biblioteka OpenCV. Zapewnia dostęp do kamery (`VideoCapture`), konwersję przestrzeni barw oraz wszelkie operacje na obrazie używane w dalszych modułach.

`threading` — standardowa biblioteka Pythona. Umożliwia uruchomienie pętli odczytu kamery w osobnym wątku, niezależnym od pętli głównej systemu.

3. Konstruktor `__init__`

```
def __init__(self, src=0, width=320, height=240, fps=30):
    self.cap = cv2.VideoCapture(src)
    self.cap.set(cv2.CAP_PROP_FRAME_WIDTH, width)
    self.cap.set(cv2.CAP_PROP_FRAME_HEIGHT, height)
    self.cap.set(cv2.CAP_PROP_FPS, fps)
    self.cap.set(cv2.CAP_PROP_BUFFERSIZE, 1)

    self.frame = None
    self.lock = threading.Lock()
    self.running = False
    self._thread: threading.Thread | None = None
```

Parametry konstruktora

src=0	Indeks kamery systemowej. 0 = pierwsza kamera (wbudowana lub USB). Przy dwóch kamerach druga ma <code>src=1</code> .
width=320	Żądana szerokość klatki w pikselach. Świadomy kompromis: 320×240 daje ~30 fps na RPi, 640×480 spada do ~15 fps.
height=240	Żądana wysokość klatki w pikselach.
fps=30	Żądana liczba klatek na sekundę. Sterownik kamery może zignorować tę wartość jeśli sprzęt nie obsługuje danego FPS.

Kluczowe ustawienia `VideoCapture`

`CAP_PROP_BUFFERSIZE = 1` — najważniejsze ustawienie. Ogranicza wewnętrzny bufor OpenCV do jednej klatki. Bez tego przy wolnym przetwarzaniu moduł zwracałby klatki sprzed setek milisekund zamiast aktualnego obrazu.

Dlaczego 320×240?

Liczba pikseli do przetworzenia rośnie kwadratowo z rozdzielczością:

320×240 = 76 800 px → ~30 fps na Raspberry Pi 4
640×480 = 307 200 px → ~15 fps na Raspberry Pi 4
1280×720 = 921 600 px → ~5 fps na Raspberry Pi 4

Dla funkcji Stop & Go minimum operacyjne to ~15 fps. Przy 5 fps pojazd poprzedzający może zdążyć zatrzymać się między klatkami.

Zmienne stanu obiektu

<code>self.frame</code>	Przechowuje ostatnią odebraną klatkę (ndarray BGR). Inicjalizowana jako None — przed pierwszą klatką.
<code>self.lock</code>	Obiekt Lock z biblioteki threading. Chroni <code>self.frame</code> przed jednoczesnym dostępem dwóch wątków (race condition).
<code>self.running</code>	Flaga bool kontrolująca pętlę wątku. False = wątek powinien się zakończyć.
<code>self._thread</code>	Referencja do obiektu Thread. Adnotacja typu eliminuje ostrzeżenia Pylance przy wywołaniu <code>join()</code> .

4. Metody — opis i działanie

`_reader()` — pętla wątku kamery

```
def _reader(self):
    while self.running:
        ok, frame = self.cap.read()
        if not ok:
            continue
        with self.lock:
            self.frame = frame
```

Metoda uruchamiana w osobnym wątku. Działa w nieskończonej pętli dopóki `self.running == True`, czytając klatki tak szybko jak kamera je produkuje.

`ok, frame = self.cap.read()` — OpenCV zwraca krotkę: flagę sukcesu i klatkę. Jeśli odczyt się nie powiódł (chwilowy drop USB, przeciążenie), `ok == False` i klatka jest pomijana.

`with self.lock:` — zamek jest zajmowany na czas nadpisania `self.frame`. Żaden inny wątek nie może odczytać klatki w połowie jej aktualizacji.

Uwaga: metoda `_reader` musi być zdefiniowana PRZED metodą `start()` w pliku.

Pylance zgłasza błąd 'Attribute `_reader` is unknown' gdy jest odwrotnie, mimo że Python w czasie wykonania nie ma z tym problemu.

`start()` — uruchomienie wątku

```
def start(self):
    self.running = True
    self._thread = threading.Thread(
        target=self._reader, daemon=True
    )
    self._thread.start()
    return self
```

`daemon=True` — wątek-demon kończy się automatycznie gdy kończy się program główny. Bez tej flagi Python czekałby na zakończenie wątku nawet po wyjściu z `main()`.

`return self` — wzorec method chaining. Pozwala pisać `cam = Camera(width=320).start()` zamiast dwóch osobnych instrukcji.

get_frame() — klatka w skali szarości

```
def get_frame(self):
    with self.lock:
        if self.frame is None:
            return None
        return cv2.cvtColor(self.frame, cv2.COLOR_BGR2GRAY)
```

Główna metoda publiczna — wywołana przez `detector.py` w każdym obiegu pętli głównej.

`cv2.COLOR_BGR2GRAY` — konwersja przestrzeni barw. Algorytmy detekcji (HOG, KCF) pracują na gradientach jasności, nie na kolorze. Skala szarości to 1 kanał zamiast 3 — trzykrotnie mniej danych dla wszystkich kolejnych operacji.

Konwersja odbywa się wewnątrz zamka — zwracany obraz jest już niezależną kopią, bezpieczną do przetwarzania przez inny wątek.

get_frame_color() — klatka BGR (do debugowania)

```
def get_frame_color(self):
    with self.lock:
        return self.frame.copy() if self.frame is not None else None
```

`.copy()` — zwracamy głęboką kopię, nie referencję do `self.frame`. Gdybyśmy zwrócili referencję, wątek kamery mógłby nadpisać dane w trakcie gdy caller jeszcze je przetwarza.

Metoda używana wyłącznie przez `draw_debug()` w module `detector.py` oraz podczas testów wizualnych. Nie jest częścią ścieżki krytycznej systemu.

stop() — bezpieczne zamknięcie

```
def stop(self):
    self.running = False
    self._thread.join()
    self.cap.release()
```

Kolejność tych trzech instrukcji jest nieprzypadkowa i nie można jej zmienić:

1. `self.running = False` — sygnalizujemy wątkowi żeby skończył pętlę przy następnym obrocie.
2. `self._thread.join()` — czekamy aż wątek faktycznie się zakończy. Bez tego moglibyśmy wywołać `cap.release()` gdy wątek jest w połowie `cap.read()` — klasyczny błąd wielowątkowy powodujący crash.
3. `self.cap.release()` — zwalniamy zasób sprzętowy. System operacyjny odblokowuje kamerę dla innych procesów. Bez tego kamera pozostaje "zajęta" nawet po zakończeniu programu.

is_ok() — diagnostyka

```
def is_ok(self):
    return self.cap.isOpened()
```

Metoda pomocnicza do weryfikacji przy starcie systemu. Używana w `main.py` przed uruchomieniem pętli głównej:

```
cam = Camera().start()
```

```
if not cam.is_ok():
    sys.exit('Błąd: kamera niedostępna')
```

5. Interfejs publiczny — podsumowanie

<code>start()</code>	Uruchamia wątek odczytu. Zwraca self (method chaining). Wywołać raz po stworzeniu obiektu.
<code>get_frame()</code>	Zwraca najświeższą klatkę GRAY (ndarray 240×320, dtype uint8) lub None. Wywołać co obieg pętli głównej.
<code>get_frame_color()</code>	Zwraca klatkę BGR (kopia). Tylko do debugowania i podglądu.
<code>stop()</code>	Zatrzymuje wątek i zwalnia kamerę. Wywołać przy zamykaniu programu.
<code>is_ok()</code>	Zwraca True jeśli kamera jest otwarta i gotowa. Używać do weryfikacji przy starcie.

6. Kompletny kod modułu

```
import cv2
import threading

class Camera:

    def __init__(self, src=0, width=320, height=240, fps=30):
        self.cap = cv2.VideoCapture(src)
        self.cap.set(cv2.CAP_PROP_FRAME_WIDTH, width)
        self.cap.set(cv2.CAP_PROP_FRAME_HEIGHT, height)
        self.cap.set(cv2.CAP_PROP_FPS, fps)
        self.cap.set(cv2.CAP_PROP_BUFFERSIZE, 1)

        self.frame = None
        self.lock = threading.Lock()
        self.running = False
        self._thread: threading.Thread | None = None

    def _reader(self):
        while self.running:
            ok, frame = self.cap.read()
            if not ok:
                continue
            with self.lock:
                self.frame = frame

    def start(self):
        self.running = True
        self._thread = threading.Thread(
            target=self._reader, daemon=True
        )
        self._thread.start()
        return self

    def get_frame(self):
        with self.lock:
            if self.frame is None:
                return None
            return cv2.cvtColor(self.frame, cv2.COLOR_BGR2GRAY)

    def get_frame_color(self):
        with self.lock:
            return self.frame.copy() if self.frame is not None else None

    def stop(self):
        self.running = False
```

```
self._thread.join()
self.cap.release()

def is_ok(self):
    return self.cap.isOpened()
```

7. Zależności i literatura

Biblioteki zewnętrzne:

opencv-python — pip install opencv-python
threading — biblioteka standardowa Python, brak instalacji

Odniesienia literaturowe:

Kaehler A., Bradski G. — Learning OpenCV 3, O'Reilly 2016
Rozdziały 2–3: VideoCapture, właściwości kamery, przestrzenie barw
Dokumentacja wybranego komputera jednopłytkowego (RPi / Jetson) —
konfiguracja GPIO, sterowniki kamery, interfejs CSI/USB